

The 8-Puzzle Problem: Hill Climbing Algorithm

AI Lab Assignment

February 3, 2026

Abstract

This document provides a mathematical treatment of the 8-puzzle problem and its solution using the Hill Climbing algorithm. We present formal definitions, algorithm pseudocode, heuristic functions, complexity analysis, and experimental results.

Contents

1	Introduction	2
1.1	Problem Statement	2
2	Heuristic Functions	2
2.1	Manhattan Distance Heuristic	2
2.2	Misplaced Tiles Heuristic	2
2.3	Dominance Relation	3
3	Hill Climbing Algorithm	3
3.1	Basic Hill Climbing	3
3.2	Hill Climbing with Sideways Moves	3
3.3	Complexity Analysis	3
3.4	Limitations	5
4	Experimental Results	5
4.1	Test Cases	5
4.1.1	Test Case 1: Simple Problem	5
4.1.2	Test Case 2: Medium Complexity	5
4.2	Performance Analysis	6
4.3	Algorithm Characteristics	6
5	Mathematical Properties	6
5.1	State Space Size	6
5.2	Branching Factor	6
6	Conclusion	7
7	References	7

1 Introduction

The 8-puzzle problem is a classic problem in artificial intelligence that involves sliding numbered tiles on a 3×3 grid to reach a goal configuration. This problem serves as an excellent testbed for studying search algorithms and heuristic functions.

1.1 Problem Statement

Definition 1 (8-Puzzle State). *An 8-puzzle state S is a 3×3 matrix where each cell contains either a number from $\{1, 2, 3, 4, 5, 6, 7, 8\}$ or a blank space (denoted as 0):*

$$S = \begin{bmatrix} s_{11} & s_{12} & s_{13} \\ s_{21} & s_{22} & s_{23} \\ s_{31} & s_{32} & s_{33} \end{bmatrix}$$

where $s_{ij} \in \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$ and each number appears exactly once.

Definition 2 (Valid Move). *A valid move m slides a tile adjacent to the blank space into the blank space. The set of valid moves from state S is:*

$$M(S) = \{m : m \in \{Up, Down, Left, Right\} \wedge \text{move } m \text{ is within bounds}\}$$

Definition 3 (State Space). *The state space $\mathcal{S}$ is the set of all reachable states from an initial state S_0 through valid moves. The state space forms a directed graph $G = (\mathcal{S}, E)$ where:*

- $\mathcal{S}$ is the set of all possible board configurations
- $E = \{(S_i, S_j) : S_j \text{ can be reached from } S_i \text{ by one valid move}\}$

2 Heuristic Functions

Heuristic functions estimate the cost from a current state to the goal state.

2.1 Manhattan Distance Heuristic

Definition 4 (Manhattan Distance). *For a state S , the Manhattan distance heuristic $h_M(S)$ is the sum of the distances of each tile from its goal position:*

$$h_M(S) = \sum_{i=1}^8 |x_i - x_i^*| + |y_i - y_i^*|$$

where (x_i, y_i) is the current position of tile i and (x_i^*, y_i^*) is its goal position.

2.2 Misplaced Tiles Heuristic

Definition 5 (Misplaced Tiles). *The misplaced tiles heuristic $h_T(S)$ counts the number of tiles not in their goal position:*

$$h_T(S) = |\{i : s_i \neq s_i^* \wedge s_i \neq 0\}|$$

where s_i is the value at position i in state S and s_i^* is the goal value.

2.3 Dominance Relation

Theorem 1 (Heuristic Dominance). *For all non-goal states S in the 8-puzzle:*

$$h_M(S) \geq h_T(S)$$

Thus, Manhattan distance dominates the misplaced tiles heuristic.

Proof. For any misplaced tile, its Manhattan distance from the goal is at least 1. Therefore:

$$h_M(S) = \sum_{i \in \text{misplaced}} d_M(i) \geq \sum_{i \in \text{misplaced}} 1 = h_T(S)$$

□

3 Hill Climbing Algorithm

Hill Climbing is a local search algorithm that iteratively moves to neighboring states with better heuristic values.

3.1 Basic Hill Climbing

Algorithm 1 Hill Climbing Search

```

1: procedure HILLCLIMBING( $S_0, S_{goal}, h$ )
2:    $current \leftarrow S_0$ 
3:    $visited \leftarrow \{S_0\}$ 
4:   while  $current \neq S_{goal}$  do
5:      $neighbors \leftarrow \text{GetSuccessors}(current) \setminus visited$ 
6:     if  $neighbors = \emptyset$  then
7:       return FAILURE
8:     end if
9:      $best \leftarrow \arg \min_{n \in neighbors} h(n)$ 
10:    if  $h(best) \geq h(current)$  then
11:      return FAILURE
12:    end if
13:     $current \leftarrow best$ 
14:     $visited \leftarrow visited \cup \{best\}$ 
15:  end while
16:  return  $current$ 
17: end procedure

```

3.2 Hill Climbing with Sideways Moves

3.3 Complexity Analysis

Theorem 2 (Time Complexity). *Hill Climbing typically explores $O(d)$ nodes where d is the number of steps taken, though it can explore $O(b^d)$ nodes in worst case where b is the branching factor.*

Algorithm 2 Hill Climbing with Sideways Moves

```
1: procedure HILLCLIMBINGSIDEWAYS( $S_0, S_{goal}, h, max_{sideways}$ )
2:    $current \leftarrow S_0$ 
3:    $visited \leftarrow \{S_0\}$ 
4:    $sideways\_count \leftarrow 0$ 
5:   while  $current \neq S_{goal}$  do
6:      $neighbors \leftarrow \text{GetSuccessors}(current) \setminus visited$ 
7:     if  $neighbors = \emptyset$  then
8:       return FAILURE
9:     end if
10:     $best \leftarrow \arg \min_{n \in neighbors} h(n)$ 
11:    if  $h(best) < h(current)$  then
12:       $current \leftarrow best$ 
13:       $sideways\_count \leftarrow 0$ 
14:    else if  $h(best) = h(current)$  and  $sideways\_count < max_{sideways}$  then
15:       $current \leftarrow best$ 
16:       $sideways\_count \leftarrow sideways\_count + 1$ 
17:    else
18:      return FAILURE
19:    end if
20:     $visited \leftarrow visited \cup \{best\}$ 
21:  end while
22:  return  $current$ 
23: end procedure
```

Theorem 3 (Space Complexity). *Hill Climbing has space complexity $O(d)$ where d is the number of steps taken.*

3.4 Limitations

Hill Climbing suffers from three main problems:

1. **Local Optima:** Getting stuck at states where all neighbors have worse heuristic values
2. **Plateaus:** Flat regions where neighbors have equal heuristic values
3. **Ridges:** Sequences of local maxima difficult to navigate

4 Experimental Results

4.1 Test Cases

We evaluated Hill Climbing on several 8-puzzle instances:

4.1.1 Test Case 1: Simple Problem

Initial State:

$$S_0 = \begin{bmatrix} 1 & 2 & 3 \\ 8 & 4 & 0 \\ 7 & 6 & 5 \end{bmatrix}$$

Goal State:

$$S_{goal} = \begin{bmatrix} 1 & 2 & 3 \\ 8 & 0 & 4 \\ 7 & 6 & 5 \end{bmatrix}$$

Results:

- Hill Climbing (Manhattan): 1 move, 1 node expanded

4.1.2 Test Case 2: Medium Complexity

Initial State:

$$S_0 = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 8 & 4 \\ 7 & 6 & 5 \end{bmatrix}$$

Goal State:

$$S_{goal} = \begin{bmatrix} 1 & 2 & 3 \\ 8 & 0 & 4 \\ 7 & 6 & 5 \end{bmatrix}$$

Results:

- Hill Climbing (Manhattan): 1 move, 1 node expanded

Heuristic	Moves	Nodes Expanded
Manhattan Distance	1	1
Misplaced Tiles	Variable	Variable

Table 1: Hill Climbing performance on simple test cases

4.2 Performance Analysis

4.3 Algorithm Characteristics

Advantages:

- Low memory requirements: $O(d)$
- Fast execution for problems without local optima
- Simple implementation

Disadvantages:

- Can get stuck at local optima
- Not guaranteed to find a solution
- Not guaranteed to find optimal solution

Improvements:

- Sideways moves: Help escape plateaus
- Random restart: Try multiple starting points
- Simulated annealing: Allow occasional uphill moves

5 Mathematical Properties

5.1 State Space Size

Theorem 4 (8-Puzzle State Space). *The total number of possible configurations in the 8-puzzle is $9! = 362,880$. However, only half are reachable from any given initial state due to parity constraints.*

Proof. There are $9!$ ways to arrange 9 tiles (including the blank). The 8-puzzle can be proven to have two disjoint classes of states based on permutation parity, making only $9!/2 = 181,440$ states reachable from any given initial state. $\square$

5.2 Branching Factor

Lemma 1 (Average Branching Factor). *The average branching factor of the 8-puzzle is approximately 2.67.*

Proof. The blank can be in one of 9 positions:

- Corner positions (4): 2 possible moves each

- Edge positions (4): 3 possible moves each
- Center position (1): 4 possible moves

Average: $\frac{4(2)+4(3)+1(4)}{9} = \frac{24}{9} \approx 2.67$

□

6 Conclusion

We have presented a mathematical treatment of the 8-puzzle problem using the Hill Climbing algorithm. Hill Climbing offers simplicity and efficiency for problems without local optima, though it can get stuck and fail to find solutions.

The algorithm is best suited for:

- Simple problems with clear gradient toward goal
- Memory-constrained environments
- Quick approximate solutions
- Cases where optimal solution is not required

Variants like sideways moves and random restart can improve performance by helping escape local optima.

7 References

1. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.